

1. How many decision points are there for a dependency parser? State them.

[2 marks]

Typically two:

- What is the relationship between a pair of tokens/words, if any? $rel(A, B)$
- Which is the head or What is the direction of the arrow? $A \rightarrow B$ or $A \leftarrow B$

2. Name and Define the actions in MaltParser. Also, provide their pre and post operation configuration of stack and buffer.

[6 marks]

Three operations:

- LeftArc_x**: Assigns relation (X) between the word (head) at the top of the stack and the word (dependent) directly beneath it. Removes the word (dependent) after processing.

Pre-Config	Stack[Root, w_{i-1} , w_i]	Buffer[w_j ,]
Post-Config (LeftArc)	Stack[Root, w_i]	Buffer[w_j ,]

- RightArc_x**: Assigns relation (X) between the second word at the top of the stack and the first word (dependent) at the top of the buffer. Removes the top of the stack word (dependent) after processing.

Pre-Config	Stack[Root, w_{i-1} , w_i]	Buffer[w_j ,]
Post-Config (RightArc)	Stack[Root, w_{i-1}]	Buffer[w_j ,]

- Shift**: Transfers the front word from buffer to top of the stack

Pre-Config	Stack[Root, w_i]	Buffer[w_j , w_{j+1} ,]
Post-Config (Shift)	Stack[Root, w_{i-1} , w_j]	Buffer[w_{j+1} ,]

3. Identify the head (governor) in the following noun phrase: "Mumbai green lights women icons"

[2 marks]

Answer: **icons**

4. Considering the following grammar, Mr. X wants to perform probabilistic CYK parsing for a sentence "I saw a girl with a telescope." However, he forgot the very first requirement of probabilistic CYK parsing.

- State the requirement that he needs to do [1 marks]

Grammar must be in CNF. Currently, $NP \rightarrow PRP$ [0.3] violates the rule. Mr. X needs to convert it into CNF.

- Explain the procedure how its done, and [2 marks]

CNF grammar productions can have two forms only.

$A \rightarrow BC$ (Two non-terminals on the right side of production)

$A \rightarrow a$ (Single terminal on the right side of production)

- Provide an appropriate solution for the given example.

[5 marks]

For the third part, the above production must go. To compensate for it, we have to introduce other productions. The easiest way is to add a production (with PRP) wherever NP is used on the right side.

Note: Probabilities are arbitrary and we don't care what value did you use as long as

$$\forall_A \sum_j P(A \rightarrow B_j) = 1$$

S	$\rightarrow$ NP VP	[1.0]
S	$\rightarrow$ PRP VP	[0.3]
NP	$\rightarrow$ DT NN	[0.5]
NP	$\rightarrow$ NP PP	[0.2]
NP	$\rightarrow$ VBD NP	[0.4]
NP	$\rightarrow$ VP PP	[0.6]
PP	$\rightarrow$ IN NP	[1.0]
PRP	$\rightarrow$ I	[1.0]
NN	$\rightarrow$ telescope	[0.5]
NN	$\rightarrow$ girl	[0.5]
VBD	$\rightarrow$ saw	[1.0]
DT	$\rightarrow$ a	[1.0]
IN	$\rightarrow$ with	[1.0]

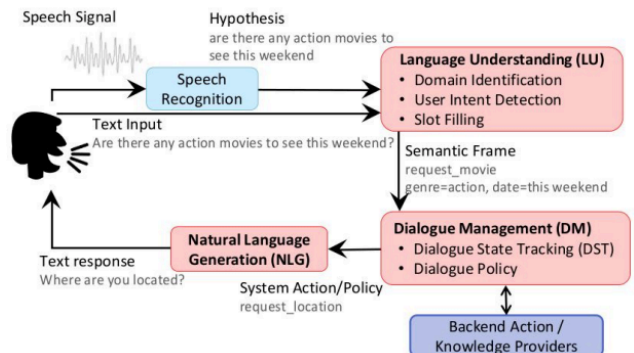
S	$\rightarrow$ NP VP	[0.9]
S	$\rightarrow$ PRP VP	[0.1]
NP	$\rightarrow$ PRP	[0.3]
NP	$\rightarrow$ DT NN	[0.5]
NP	$\rightarrow$ NP PP	[0.2]
NP	$\rightarrow$ PRP PP	[0.3]
VP	$\rightarrow$ VBD NP	[0.3]
VP	$\rightarrow$ VBD PRP	[0.1]
VP	$\rightarrow$ VP PP	[0.6]
PP	$\rightarrow$ IN NP	[0.9]
PP	$\rightarrow$ IN PRP	[0.1]
PRP	$\rightarrow$ I	[1.0]
NN	$\rightarrow$ telescope	[0.5]
NN	$\rightarrow$ girl	[0.5]
VBD	$\rightarrow$ saw	[1.0]
DT	$\rightarrow$ a	[1.0]
IN	$\rightarrow$ with	[1.0]

5. State the motivation for Pointer Generator Network (PGN) [See et al., 2017, ACL]. **[3 marks]**
 In the summarization task, we want to preserve the factual details intact. However, a typical generative model learns an output distribution over the vocabulary, which may cause inaccurate details. Therefore, instead of generating a factual detail, PGN prioritizes outputting tokens from input distribution.
6. Provide a graphical overview (e.g., block-diagram, etc.) of a complete dialog system. **Note:** It should include all major components/subcomponents. Additionally, edges should be appropriately annotated. **[5 marks]**

Should be in-line with the following diagram (Lecture 12:

Conversational Dialog Slide - 8).

- Must have components:
 - (Natural) Language Understanding
 - Intent, Slot, Affect etc.
 - Dialog Management
 - State Tracking, Policy Learning, etc.
 - Backend/API calls.
 - Natural Language Generation
- ASR and TTS may be omitted.



7. Define continued pre-training and state how it is different from pre-training. **[2 marks]**

Lecture 11: HateSpeech Slide-10

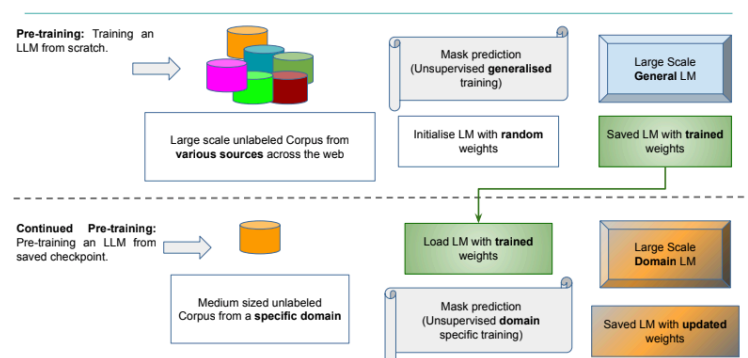
Pretraining:

- Take some large-scale generic unlabelled data
- Initialize the model parameters randomly
- Pre-train a model using MLM or other objective functions
- Save weights of the model.

Continued pretraining:

- Take medium-size domain-specific unlabelled data
- Initialize the model parameters using the saved weights after pre-training
- Continue pre-training using MLM or other objective functions
- Save updated model

Continued Pre-training: Domain specific



8. Comparison among models should be fair. State the two basic requirements to ensure fair comparison. **[2 marks]**

Lecture 9: Evaluation Slide-9

- All models should be evaluated/inferenced on exactly the same test data.
- Input to all models must be the same.

There could be other answers as well. We will evaluate them based on merits.

9. The input and output dimensions of a self-attention block may not necessarily be the same. Support or refute the claim with proper justification in terms of pros and cons. **[4 marks]**

The statement is true and there is no theoretical limitation on the sizes of input and output dimensions. However, if the output dimension is always smaller than the input dimension, there is a limit on the number of stacking of attention blocks – after a few steps, the input dimension may become 1 and we cannot further expand it to the other layer. This drawback will pose a challenge with modern systems which use 10s to 100s of stacked layers.

For simplicity and to avoid unnecessary burden, we use the same dimensions for both input and output – thus allowing us to have any number of stacked layers.

10. Explain SMT using equations. Describe the challenges in computing it and suggest a typical solution. **[10 marks]**

Statistical MT (SMT): Given a source sentence s , we compute the conditional probability of target sentence t as:

$$P(t|s) = \operatorname{argmax}_t P(s|t) P(t)$$

where $P(s|t)$ is the translation model and $P(t)$ is the language model

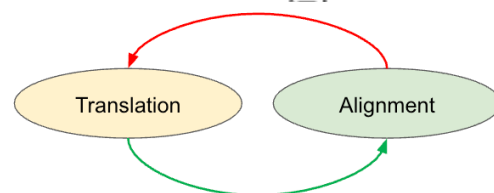
To compute the translation model $P(s|t)$, we need the probability of aligned target words, which can be computed through an alignment model $P(a)$.

$$P(s|t) = P(a) \prod_{i=1}^m P(s_i|t_{a_i})$$

$$P(a) = \prod_{i=1}^m P(s_i|t_{a_i})$$

To compute the alignment model, we need word-level translation probabilities. This creates a chicken-and-egg problem.

Expectation Maximization (EM) algorithm is a typical solution for computing the translation and alignment model over a few iterations.



11. Mr. X is designing a classification system and he has to make a choice between CNN and RNN with following details:

- Input size for both models: $100 \times n \times 300 \Rightarrow 100$ sentences with n tokens each having embedding size 300.
- RNN:
 - Recurrent dimension: 100
 - Classification head: 2 layers with dimensions 100 and 2, respectively.
- CNN:
 - Kernels: Hundred 5-gram, 4-gram, 3-gram kernels each with stride 2.
 - No pooling.
 - Classification head: 2 layers with dimensions 100 and 2, respectively.

- a. Help him decide which would be better considering the number of parameters to learn. **[1 marks]**
 b. Report the number of parameters for both models with layer/recurrent/kernel wise segregation. Ignore the bias. **[5]**

RNN - Input dimension for each token: 300 - Recurrent dimension: 100 - Classification Head dimension 1: 100 - Classification Head dimension 2: 2 Input-to-Recurrent weights: $300 \times 100 = 30000$ Recurrent-to-Recurrent weights: $100 \times 100 = 10000$ Recurrent-to-Classification Head 1 weights: $100 \times 100 = 10000$ Classification Head 1 -to- Classification Head 2 weights: $100 \times 2 = 200$ Total: 50200	CNN - Input dimension for the sentence token: $n \times 300$ - Kernels: 100 5-gram filter weights: $100 \times 5 \times 300 = 150000$ 100 4-gram filter weights: $100 \times 4 \times 300 = 120000$ 100 3-gram filter weights: $100 \times 3 \times 300 = 90000$ - Convolved features dimension after flattened (These are not weights): $100 * [(n - 5 / 2) + 1] + (n - 4 / 2) + 1 + (n - 3 / 2) + 1$ $\Rightarrow$ Let say, these are m - Convolved features-to-Classification Head 1 weights: $m * 100$ - Classification Head 1 -to- Classification Head 2 weights: $100 \times 2 = 200$ Total: $260000 + (m * 100) + 200$
--	---

So the winner is RNN.

$$\text{Convolved features dimension} = \left\lfloor \frac{\text{input-size} - \text{kernel-size}}{2} \right\rfloor + 1$$

Note for TAs:

For CNN, if the calculations are representative (or algebraic) and conveys the value correctly, accept it.